

Aula 1: Introdução a Linguagem C

Profa.: Andréia Rodrigues Casare

E-mail: casareandreia@gmail.com

andreia.casare@cps.sp.gov.br

Estrutura básica de um programa C

Diretivas

Declarações de variáveis/constantes globais

Prototipação de funções

tipo_dado função main(lista de parâmetros)

{

declarações de variáveis/constantes locais;

comandos;

estruturas de controles;

comentários;

chamada de funções;

}

tipo_dado função1(lista de parâmetros)

{

Bloco de comandos;

}

Comentários no Programa

- Comentários são pedaços de código-fonte descartados do código pelo compilador. Eles não fazem nada. O objetivo deles é somente permitir que o programador insira notas ou descrições dentro do código-fonte.
- **// comentário de linha**
- **/* comentário de bloco */**

Tipos básicos de dados

char = caracter

int = inteiro

float = ponto flutuante

double = ponto flutuante de precisão dupla

void = sem valor

Tipos de dados modificados

Tipo	Num de bits	Formato para leitura com scanf	Intervalo	
			Início	Fim
Char	8	%c	-128	127
unsigned char	8	%c	0	255
Signed char	8	%c	-128	127
Int	16	%i ou %d	-32.768	32.767
unsigned int	16	%u	0	65.535
signed int	16	%i ou %d	-32.768	32.767
Short int	16	%hi ou %hd	-32.768	32.767
unsigned short int	16	%hu	0	65.535
signed short int	16	%hi ou %hd	-32.768	32.767
Long int	32	%li ou %ld	-2.147.483.648	2.147.483.647
signed long int	32	%li ou %ld	-2.147.483.648	2.147.483.647
unsigned long int	32	%lu	0	4.294.967.295
Float*	32	%f, %e ou %g	3,4E-38	3.4E+38
Double*	64	%lf, %le ou %lg	1,7E-308	1,7E+308
long double*	80	%Lf	3,4E-4932	3,4E+4932

Constantes

- **Variáveis do tipo *const* não podem ser modificadas pelo programa, podendo, entretanto, receber um valor inicial.**
 - ***const float pi = 3.14;***

Declaração de variáveis

- Todas as variáveis do C devem ser declaradas antes de serem usadas. A forma geral de uma declaração é:
 - ***tipo lista_de_variáveis;***
- Sendo:
 - ***Tipo***: um tipo de dado válido em C mais quaisquer modificadores; e
 - ***Lista_de_variáveis***: um ou mais nomes de identificadores separados por vírgula.

Operadores

<i>Operador</i>	<i>Finalidade</i>
+	Soma (inteira e ponto flutuante)
-	Subtração ou Troca de Sinal (inteira e ponto flutuante)
*	Multiplicação (inteira e ponto flutuante)
/	Divisão (inteira e ponto flutuante)
%	Resto da Divisão (somente de inteiros)

Operadores de incremento e decremento

- A linguagem C permite a utilização dos operadores unários “++” e “--” para incrementar (somar 1) ou decrementar (subtrair 1) o conteúdo de uma variável, respectivamente. Exemplos de sua utilização:
- `++X; /* Equivalente à expressão  $X = X + 1$  */`
- `X++; /* Equivalente à expressão  $X = X + 1$  */`
- `--X; /* Equivalente à expressão  $X = X - 1$  */`
- `X--; /* Equivalente à expressão  $X = X - 1$  */`

Operadores de incremento e decremento

- Estes operadores podem ser pré-fixados ou pós-fixados. A diferença é que quando são pré-fixados, eles incrementam a variável antes de usar seu valor. Quando são pós-fixados, eles utilizam o valor da variável antes de incrementá-la.

int X= 10, Y;		int X=10, Y;
Y=X++;		Y=++X;
Neste caso y=10 e X = 11		Neste caso y e X = 11

Operadores relacionais

<i>Operador</i>	<i>Finalidade</i>
==	Igual
>	Maior
<	Menor
>=	Maior e igual
<=	Menor e igual
!=	Diferente

Operadores lógicos

<i>Operador</i>	<i>Finalidade</i>
&&	AND (E)
 	OR (OU)
!	NOT (NÃO)

Comandos de entrada e saída – printf()

- A função **printf()** possibilita a saída de valores (constantes, variáveis ou resultados de expressões). Esta função converte e imprime seus argumentos na saída padrão seguindo o formato especificado na **string de controle**. Sua sintaxe geral é:
 - **printf(“string de controle”, lista de argumentos);**
- A **string de controle**, delimitada por aspas, descreve tudo que a função irá colocar na tela. Esta string não mostra apenas os caracteres que devem ser apresentados, mas também os respectivos formatos e posições das constantes, variáveis e expressões listadas na **lista de argumentos**.

Alguns especificadores de formato da função printf()

Especificador	Tipo do Argumento	Descrição
%d	int	Valor inteiro decimal
%i	int	Valor inteiro decimal
%o	int	Valor inteiro octal
%x	int	Valor inteiro hexadecimal
%X	int	Valor inteiro hexadecimal
%u	int	Valor inteiro decimal sem sinal (<i>unsigned int</i>)
%c	int	Um caracter em formato ASCII (código binário correspondente)
%s	char	Uma cadeia de caracteres (<i>string</i>) terminada em "\0"
%f	float	Valor em ponto flutuante no formato [-]m.dddddd, onde o N° de d's é dado pela precisão (padrão é 6)
%e	float ou double	Valor em ponto flutuante em notação exponencial no formato [-]m.dddddd e±xx, onde o N° de d's é dado pela precisão
%E	float ou double	Valor em ponto flutuante em notação exponencial no formato [-]m.dddddd E±xx, onde o N° de d's é dado pela precisão
%g	float ou double	Valor em ponto flutuante no formato %e (quando o expoente for menor que -4 ou igual a precisão) ou %f (nos demais casos). Zeros adicionais e um ponto decimal final não são impressos
%G	float ou double	Valor em ponto flutuante no formato %E (quando o expoente for menor que -4 ou igual a precisão) ou %f (nos demais casos). Zeros adicionais e um ponto decimal final não são impressos
%%	-	Exibe o caracter '%'

Comandos de entrada e saída – `scanf()`

- A função ***scanf()*** lê caracteres da entrada padrão (teclado), interpreta-os segundo a especificação de formato e armazena os resultados em variáveis declaradas no programa. Sintaxe:
- ***scanf*** (“*string de controle*”, *lista de endereços de memória das variáveis*);
- **A representação do endereço de memória das variáveis** que receberão os dados de entrada é fornecido **pelo operador “&”, seguido pelo nome da variável**. Além disso, assim como na função ***printf()***, esta função também utiliza os caracteres especiais de controle via códigos de barra invertida.

Alguns especificadores de formato da função scanf()

Especificador	Descrição
%d	Indica um valor inteiro decimal
%i	Indica um valor inteiro (podendo estar na base decimal, octal com inicial 0 (zero) ou hexadecimal com inicial 0X)
%u	Indica um valor inteiro decimal sem sinal
%c	Indica um caracter (<i>char</i>)
%s	Indica uma <i>string</i>
%f, %e, %g	Indica um valor em ponto flutuante de precisão simples (<i>float</i>)
%lf, %le, %lg	Indica um valor em ponto flutuante de precisão dupla (<i>double</i>)
%o	Indica um valor inteiro octal (com ou sem zero inicial)
%x ou %X	Indica um valor inteiro hexadecimal (com ou sem "0x" inicial)

Exemplo

```
# include <stdio.h>
int main(void)
{
    float SOMA, NRO;
    int CONT;
    CONT = SOMA = 0;
    printf ("\n Entre com os números a serem somados ou digite CTRL-Z para terminar \n");
    while (scanf("%f",&NRO) == 1)
    {
        CONT++;
        if (CONT == 10)
        {
            printf("\t Subtotal: %.2f \n", SOMA += NRO);
            CONT = 0;
        }
        else{
            SOMA += NRO;
        }
        printf ("\t Total: %.2f \n", SOMA);
    }
}
```

Declarações de variáveis – exemplos

- `int a,b, c;`
- `float n1, n2;`
- `double x, y, media, soma;`

#include <iostream>

- Comandos que começam com um sinal de numeração (#) são diretivas do préprocessador.
- Elas não são linhas de código executáveis, mas indicações para o compilador.
- Nesse caso, o comando `#include <iostream>` diz ao pré-processador do compilador para incluir o arquivo de cabeçalho padrão `iostream`. Esse arquivo específico inclui as declarações da biblioteca básica de entrada-saída do C++.

Impressão de códigos especiais

Código	Ação
\n	Leva o cursor para a próxima linha
\t	Executa uma tabulação
\b	Executa um retrocesso
\f	Leva o cursor para a próxima página
\a	Emite um sinal sonoro (<i>beep</i>)
\"	Exibe o caractere "
\\	Exibe o caractere \
% %	Exibe o caractere %

Exemplos definindo a quantidade casas decimais

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Default: %f \n",3.1415169265);
```

```
    printf("Uma casa: %.1f \n",3.1415169265);
```

```
    printf("Duas casas: %.2f \n",3.1415169265);
```

```
    printf("Três casas: %.3f \n",3.1415169265);
```

```
    printf("Notação Científica: %e \n",3.1415169265);
```

```
}
```

A função main (principal)

- Todo programa em C/C++ existe uma função denominada *main*.
- A função main é o ponto de entrada do programa.
- Estrutura padrão

```
int main()  
{  
    printf("olá mundo");  
}
```

Acentuação

- Para conseguir acentuar no C, deve ser usado a biblioteca `locale.h` com a função `setlocale`.

- Exemplo:

```
#include <stdio.h>
#include <locale.h>
int main(){
    setlocale(LC_ALL, "Portuguese");
    printf("olá mundo !!");
    printf("\n ç é â");
    return 0;
}
```

Exemplo 1

```
#include <stdio.h>

int main(){
    float n1,n2,media;
    printf("\n Digite a nota 1:");
    scanf("%f",&n1);
    printf("\n Digite a nota 2:");
    scanf("%f",&n2);
    media=(n1+n2)/2;
    printf("\n A média é: %.2f",media);
    return 0;
}
```

Exemplo 2

```
#include <stdio.h>
```

```
int main(){  
    float sal, novosal;  
    printf("\n Digite o salário:");  
    scanf("%f",&sal);  
    novosal= sal + sal * 25/100;  
    printf("\n Novo salário: %.2f",novosal);  
    return 0;  
}
```

Exercício

1. Crie um programa que receba dois números inteiros. Calcule:
 - Acrescente mais 10 ao primeiro número;
 - Acrescente mais 5 ao segundo número.
 - Calcule o produto desses dois números e exiba na tela o resultado.